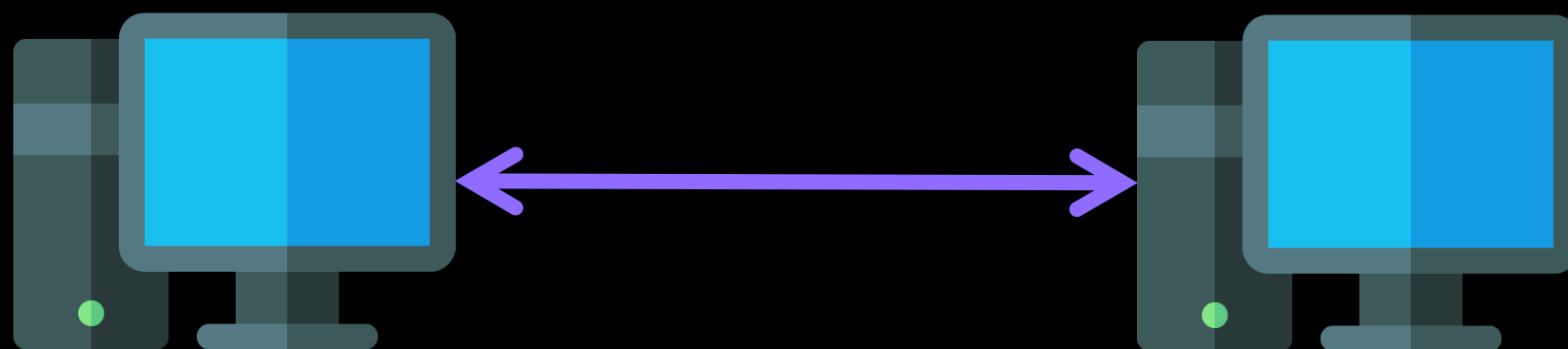


비동기 III – Ajax와 RestAPI

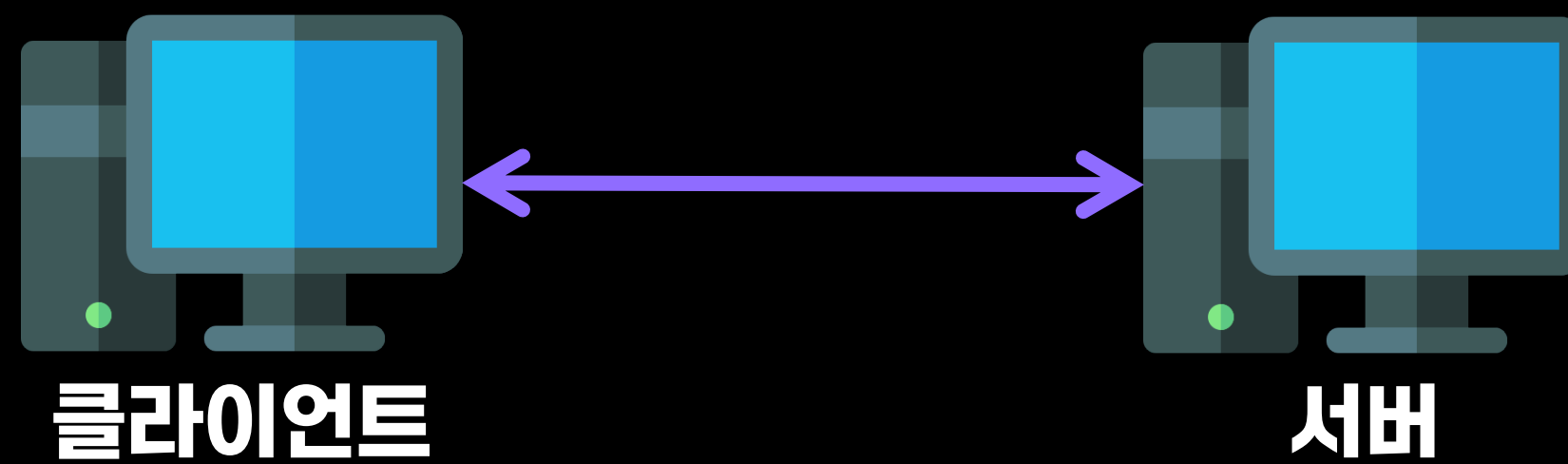


웹 애플리케이션의 구조





웹 애플리케이션의 구조



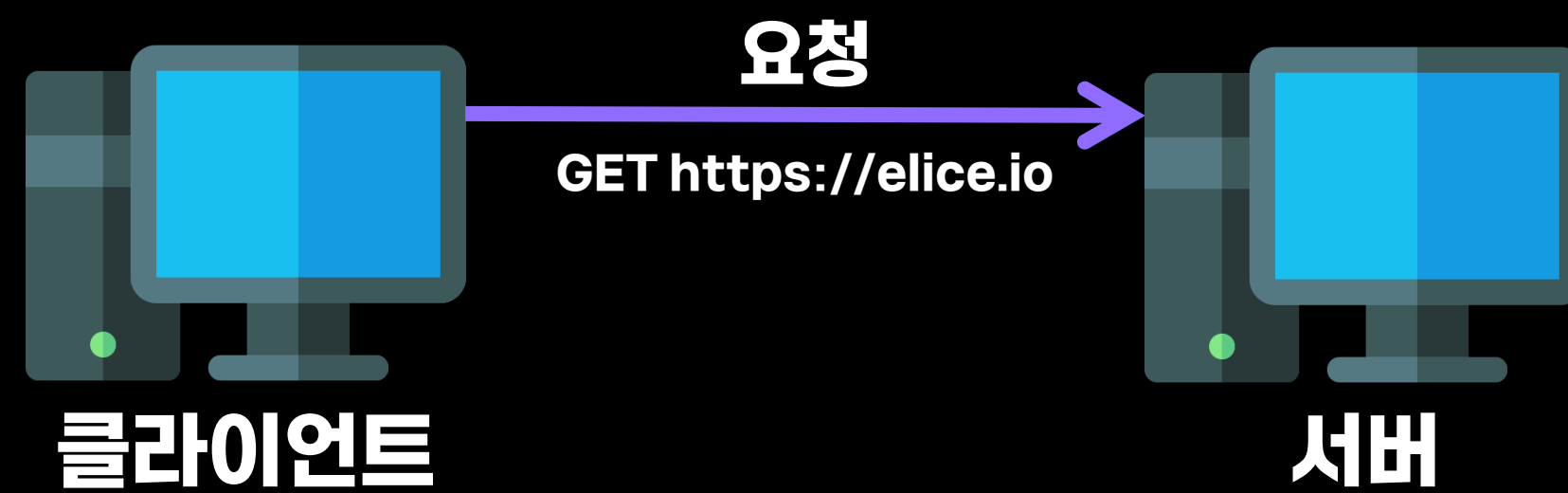


웹 애플리케이션의 구조



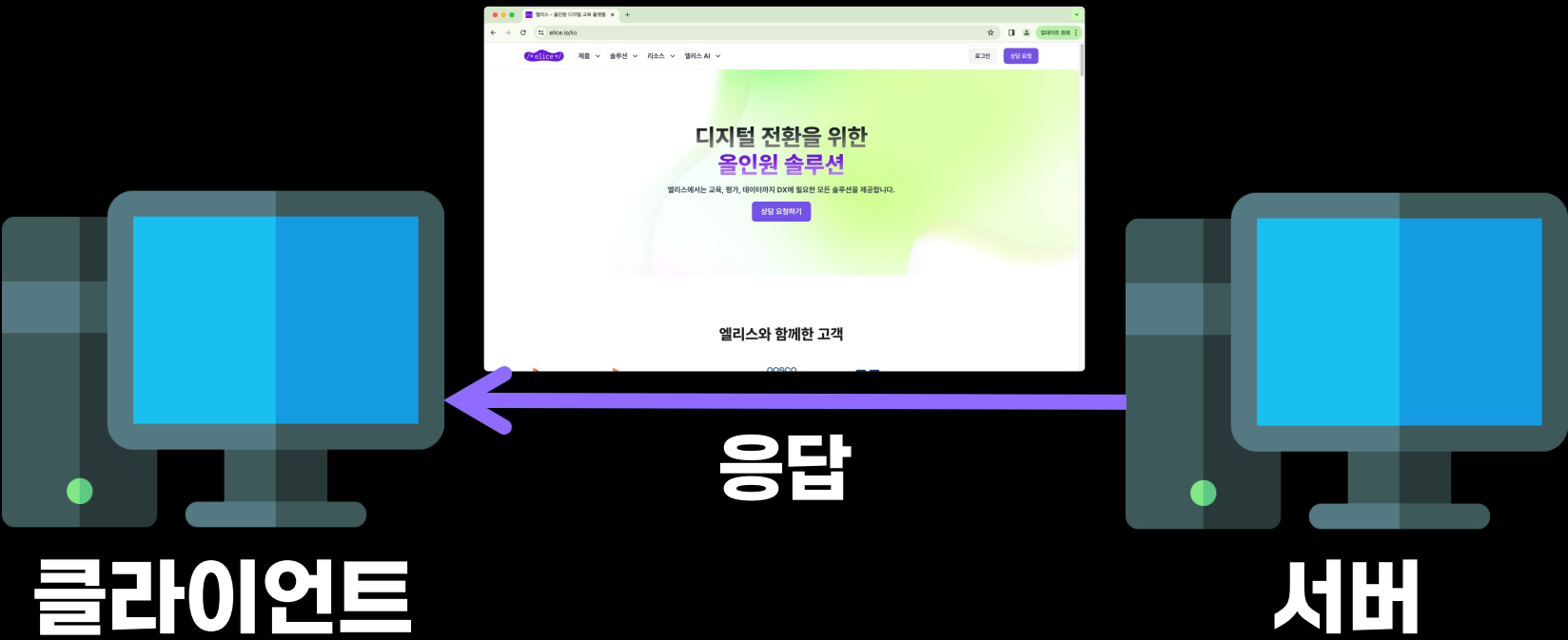


웹 애플리케이션의 구조



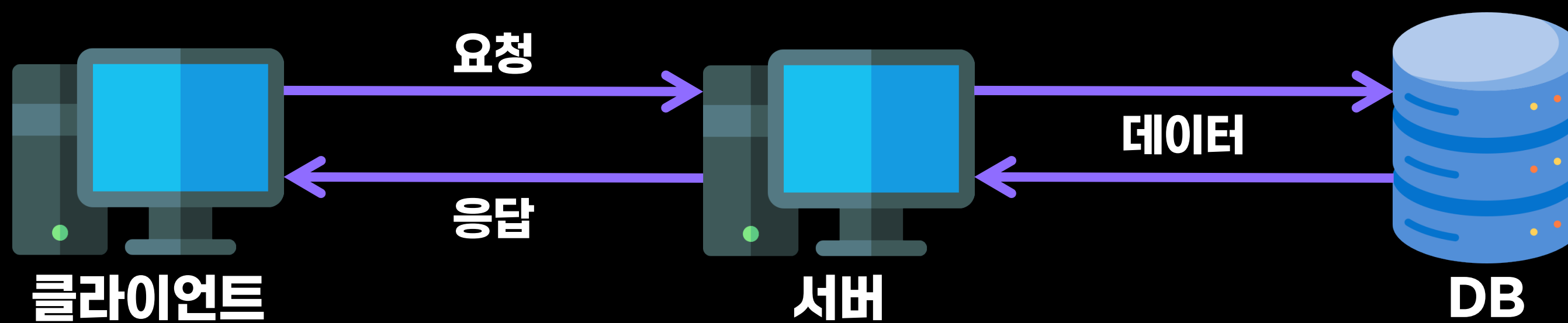


웹 애플리케이션의 구조



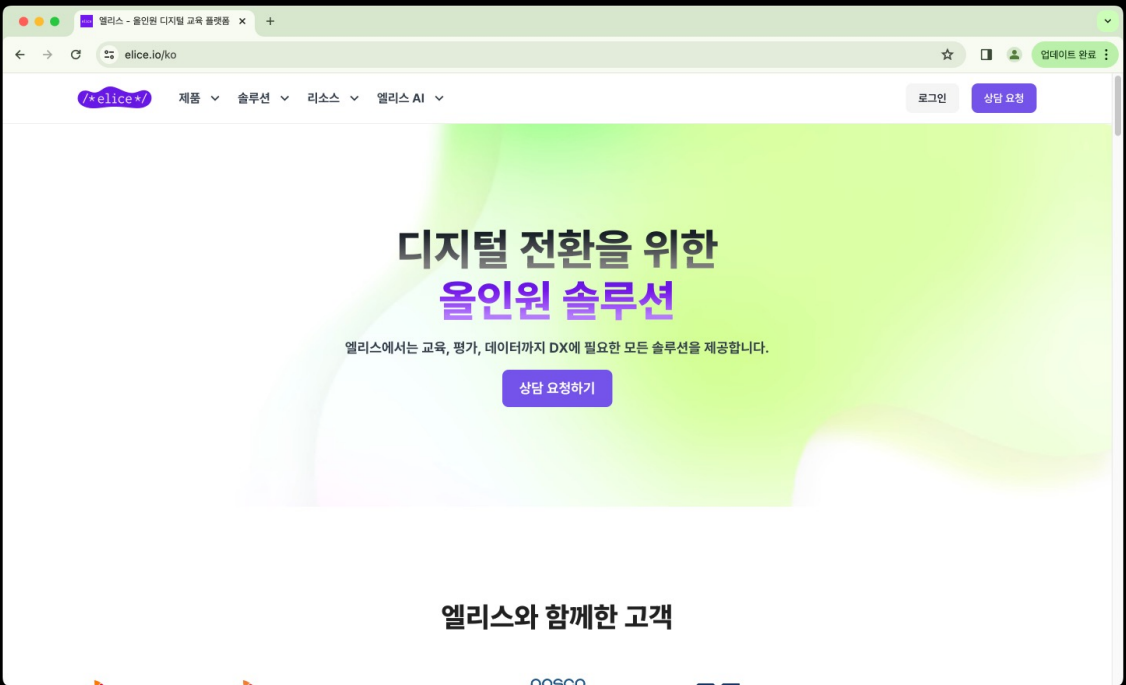


+ 데이터베이스

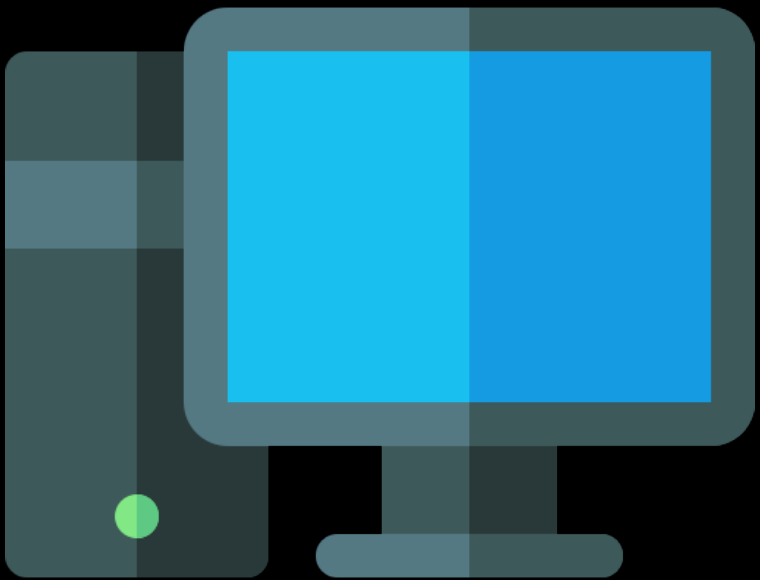




프론트엔드와 백엔드



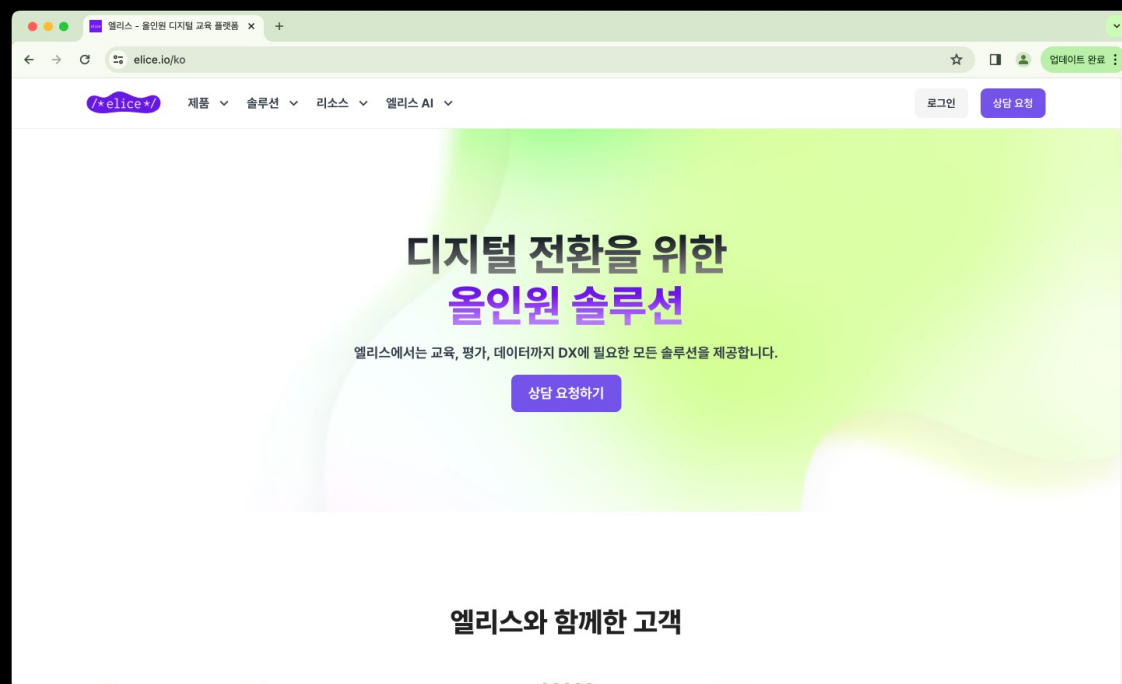
프론트엔드



백엔드

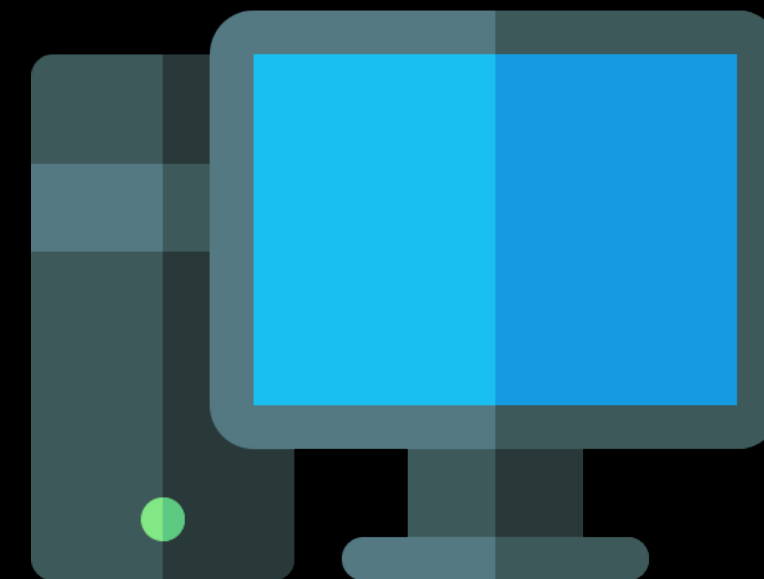


프론트엔드와 백엔드



프론트엔드

클라이언트(브라우저)에서 렌더링 되는
화면과 동작을 개발하는 것.
서버에서 데이터를 전달받아 화면에 보여주거나,
서버에 데이터 변경을 요청한다.



백엔드

클라이언트에서 요청하는 데이터를 전달하거나,
클라이언트의 요청에 맞게 데이터 변경을 진행하는
프로그램을 개발하는 것.
백엔드 프로그램은 서버에서 끊임없이 돌아간다.



프론트엔드와 백엔드



프론트엔드



백엔드



프론트엔드와 백엔드



프론트엔드



백엔드



서버에 어떻게 데이터를 요청하고,
어떻게 데이터를 전달 받을 수 있을까?





서버에 어떻게 데이터를 요청하고,
어떻게 데이터를 전달 받을 수 있을까?

HTTP

HyperText Transfer Protocol



HTTP

HyperText Transfer Protocol

클라이언트-서버 간 데이터가 교환되는 방식에 대한 규약
거의 모든 형태의 데이터 전송 가능
무상태성과 비연결성



HTTP 프로토콜의 특징

무상태성 (Stateless)

서버가 클라이언트의 상태를 보존하지 않음

클라이언트의 요청은 독립적으로 처리되며,

서버는 요청에 대한 응답만 전송함

서버의 확장성에 용이

비연결성(Connectionless)

클라이언트-서버 간 연결을 유지하지 않음

서버의 불필요한 연결을 줄이고 많은

클라이언트의 요청을 동시 처리 가능

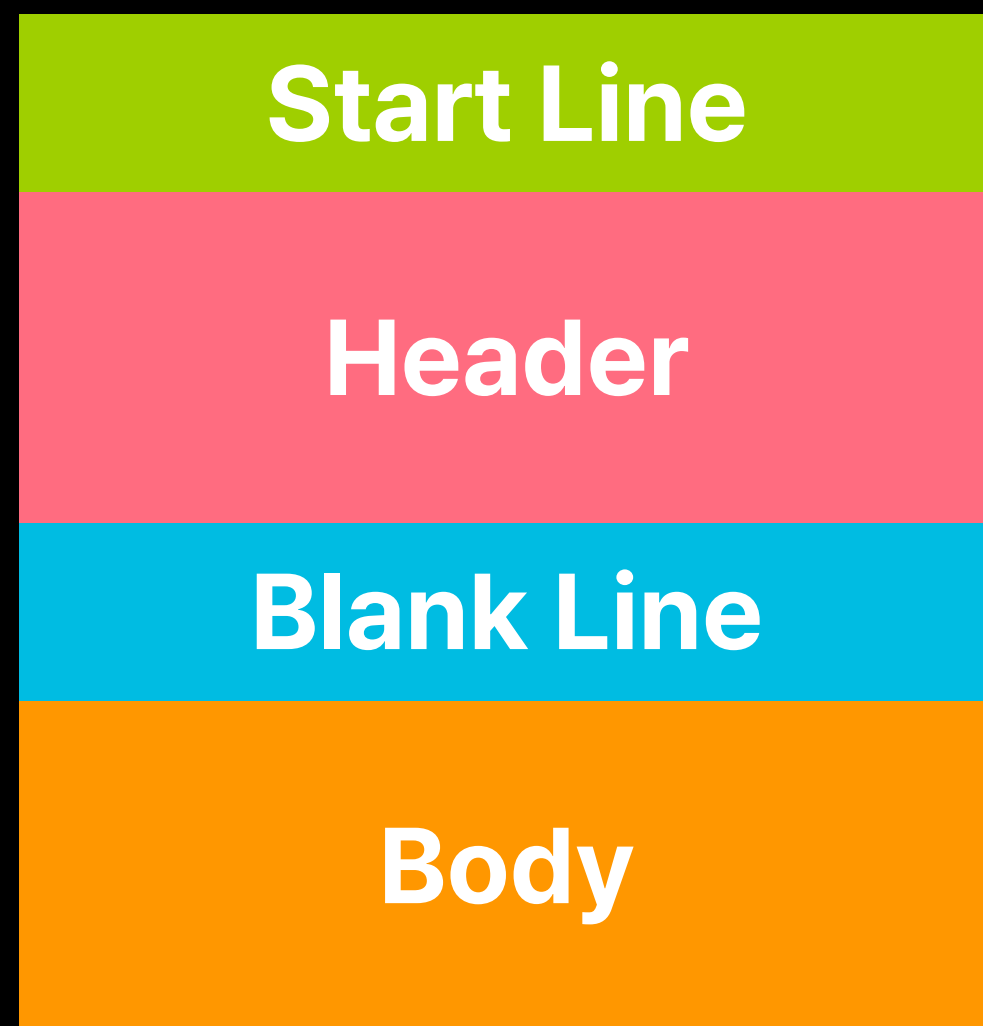


프로토콜의 구성 요소

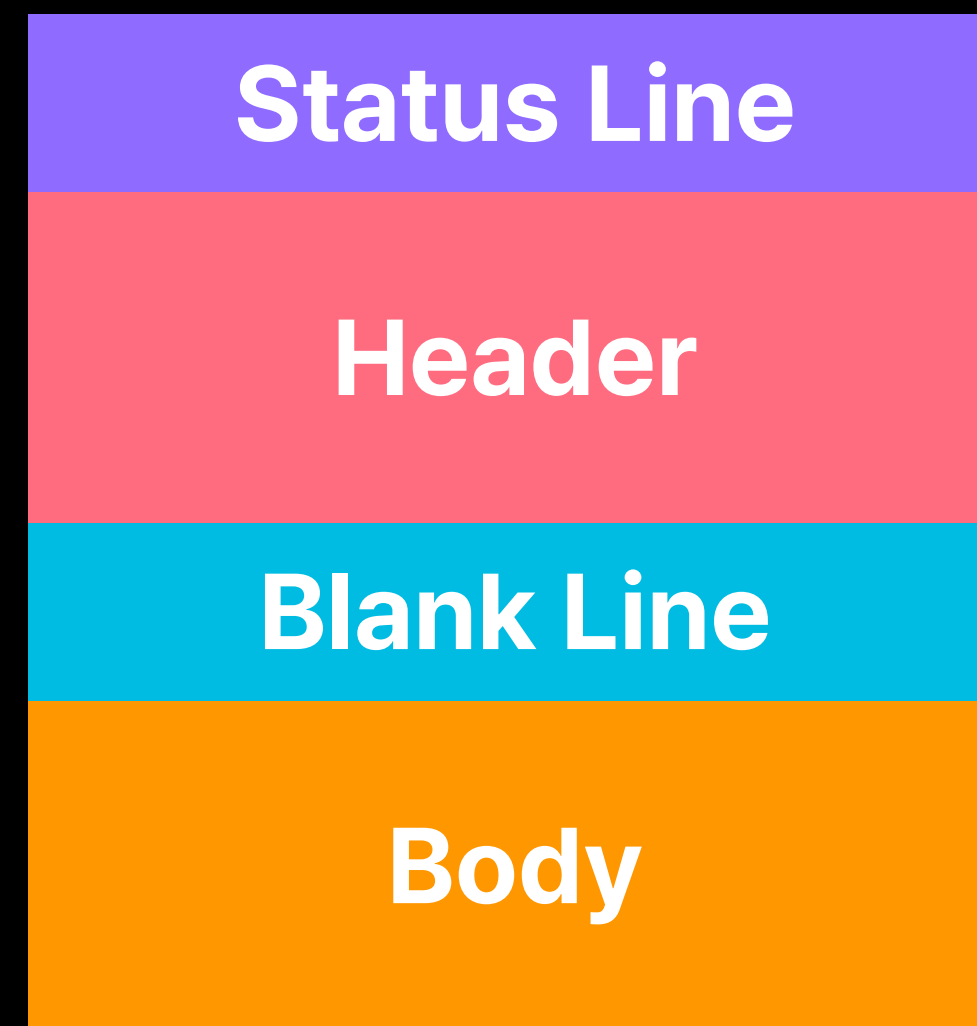
구분	의미
구문 (Syntax)	전송하고자 하는 데이터의 형식, 부호화, 신호 레벨 등을 규정
의미 (Semantics)	효율적이고 정확한 정보 전송을 위한 제어 정보를 규정
시간 (Timing)	통신 속도, 순서 제어 등을 규정



HTTP 메시지 구조



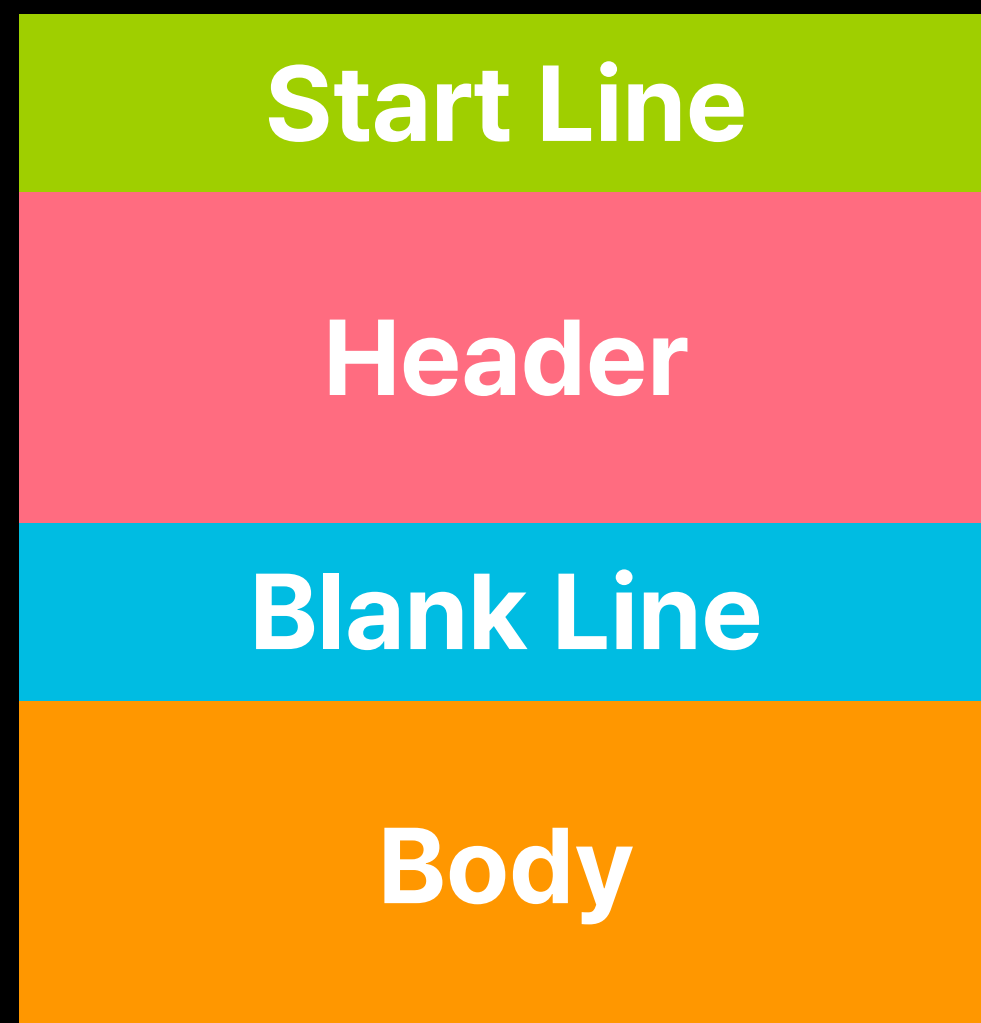
Request Message



Response Message



HTTP 요청 메시지



Request Message

```
POST /api/data HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...
Accept: application/json
Content-Type: application/json
Content-Length: 75

{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "age": 30
}
```




HTTP 요청 메시지

Start Line

Header

Blank Line

Body

Request Message

POST /api/data HTTP/1.1



HTTP 요청 메시지



Request Message

POST /api/data HTTP/1.1



HTTP 메서드

클라이언트가 서버에 요청하는 작업의 종류를 나타낸다.

HTTP 메서드	역할
GET	데이터 조회
POST	데이터 생성
PUT	데이터 전체 교체
PATCH	데이터 일부 수정
DELETE	데이터 삭제



HTTP 요청 메시지



Request Message

POST **/api/data** HTTP/1.1



요청 타겟

클라이언트가 접근하고자 하는 자원의 경로를 나타낸다.
주로 URL로 표현된다.



HTTP 요청 메시지



Request Message

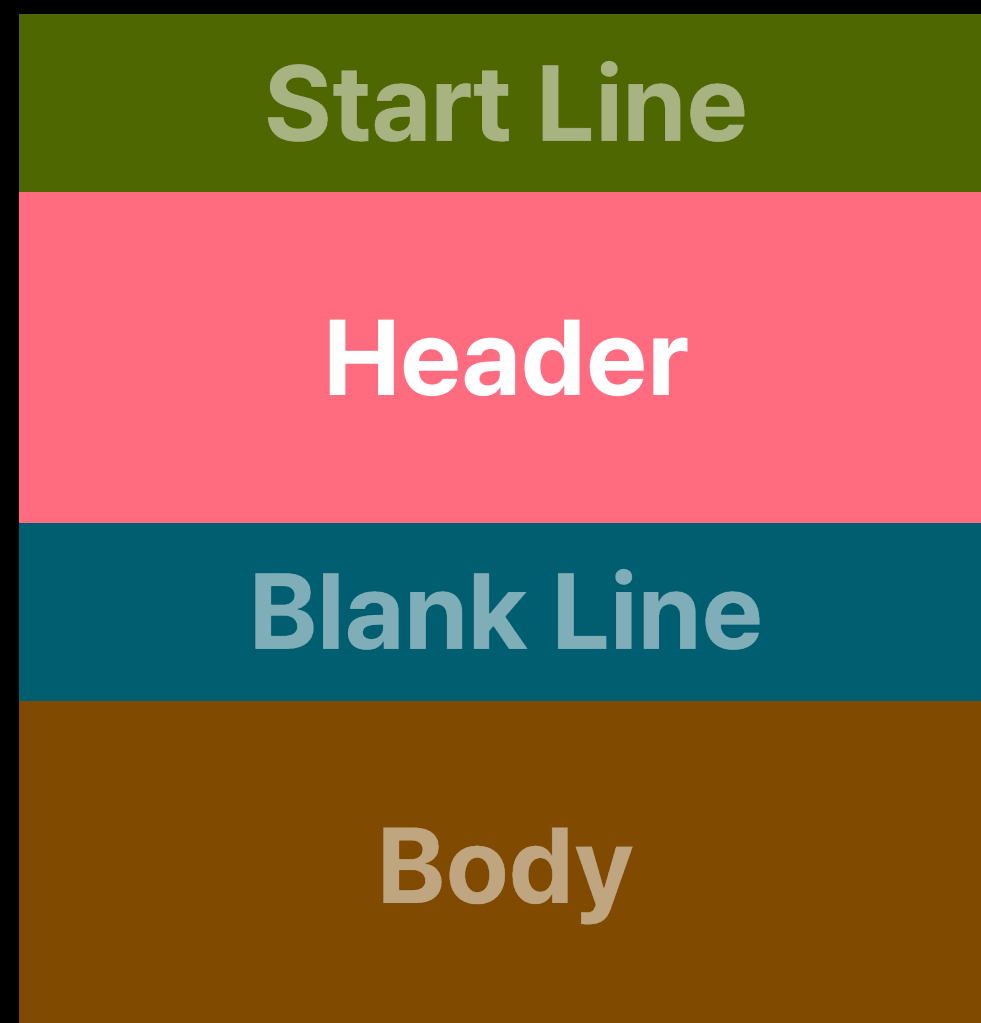
POST /api/data **HTTP/1.1**



HTTP 버전



HTTP 요청 메시지



Request Message

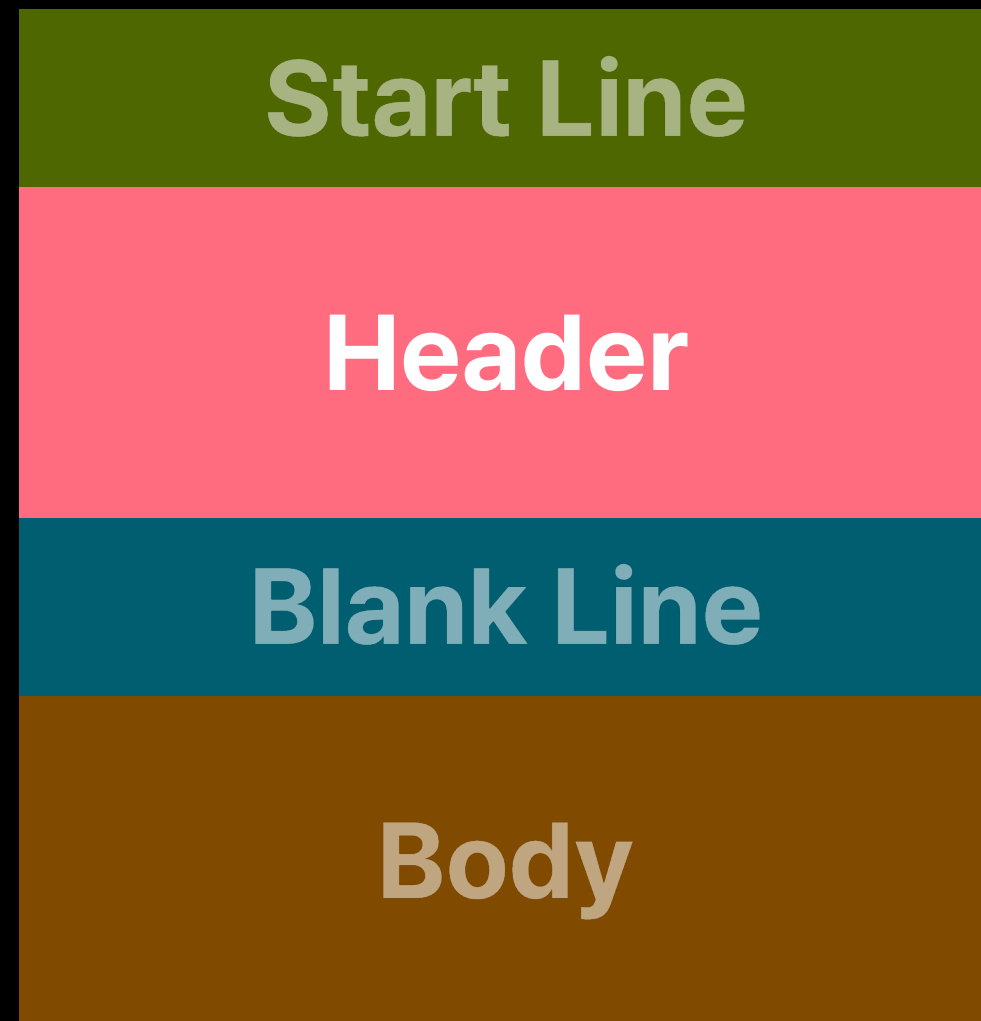
```
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...
Accept: application/json
Content-Type: application/json
Content-Length: 75
```

Header

헤더는 요청에 대한 추가 정보를 제공한다.



HTTP 요청 메시지



Request Message

```
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...
Accept: application/json
Content-Type: application/json
Content-Length: 75
```



Content-Type

Body가 존재하는 경우, Body에 전달되는 데이터의 형식을 설명한다.

ex) application/json, text/html, multipart/form-data



HTTP 요청 메시지



Request Message

Body

클라이언트가 서버에 전달하는 **실제 데이터**가 포함되며, 헤더에 설정된 Content-Type에 맞는 포맷으로 작성된다.
설정된 HTTP 메서드에 따라 없을 수도 있으며,
주로 **POST, PUT 요청**에 사용된다.

```
{  
  "name": "John Doe",  
  "email": "john.doe@example.com",  
  "age": 30  
}
```




HTTP 응답 메시지

Status Line

Header

Blank Line

Body

Response Message

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2023 12:28:53 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: application/json
Content-Length: 85
```

```
{
  "status": "success",
  "data": {
    "id": 123,
    "name": "John Doe",
    "email": "john.doe@example.com"
  }
}
```



HTTP 응답 메시지

Status Line

Header

Blank Line

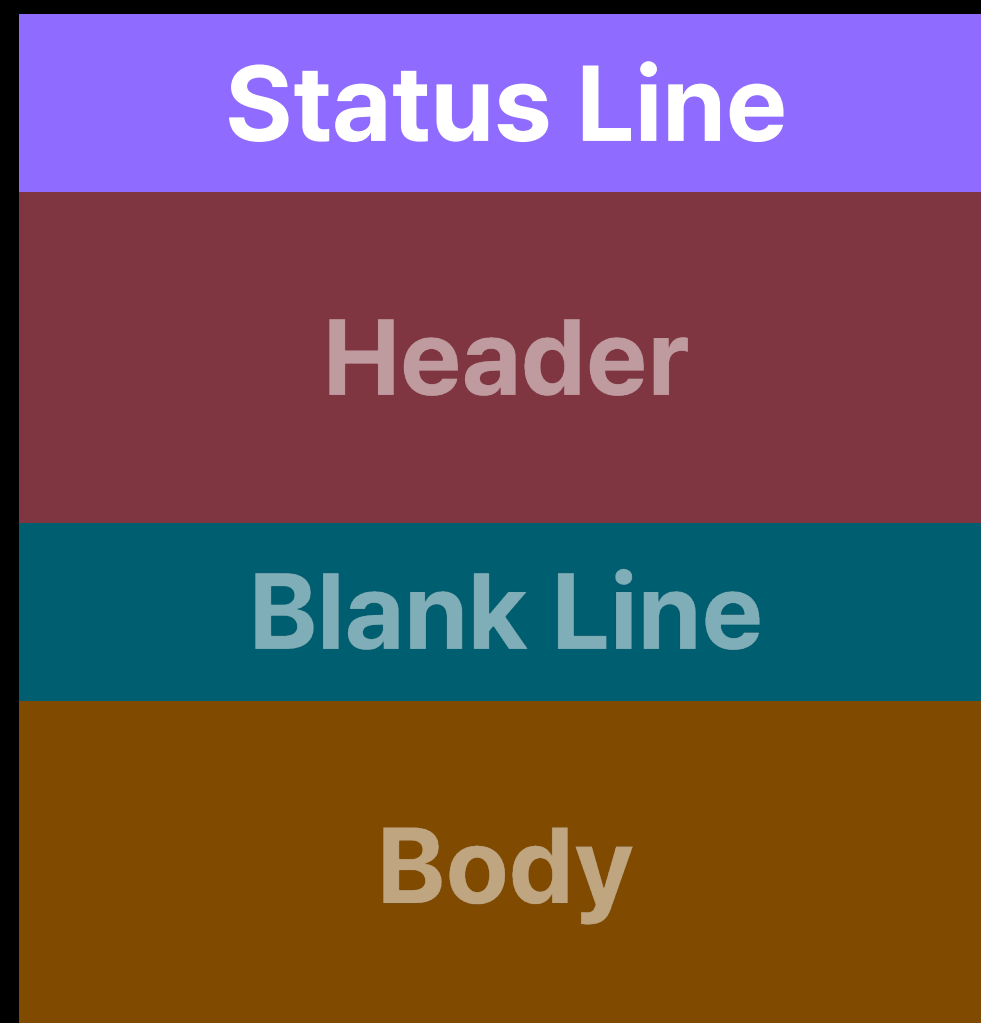
Body

Response Message

HTTP/1.1 200 OK

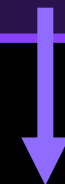


HTTP 응답 메시지



Response Message

HTTP/1.1 **200** OK



상태 코드

요청의 성공 여부를 나타낸다.

세 자리 숫자이며, 같은 범위의 코드는 비슷한 의미를 가지지만 세부적인 사유는 다를 수 있다.

- 1xx : Information (정보)
- 2xx : Successful (성공)
- 3xx : Redirection (리다이렉션)
- 4xx : Client Error (클라이언트 에러)
- 5xx : Server Error (서버 에러)



API란?

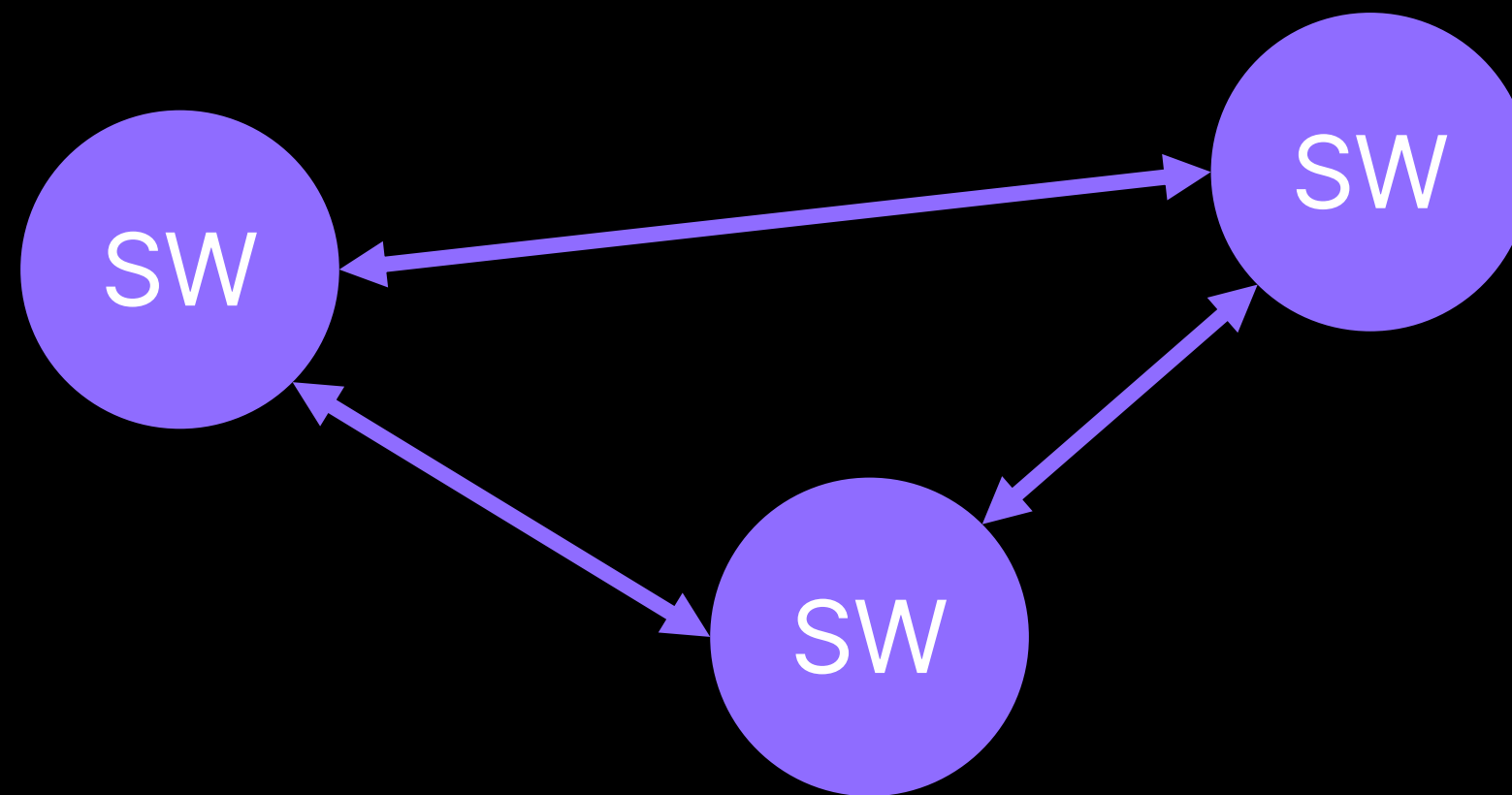
API

Application **P**rogramming **I**nterface



API

Application **P**rogramming **I**nterface

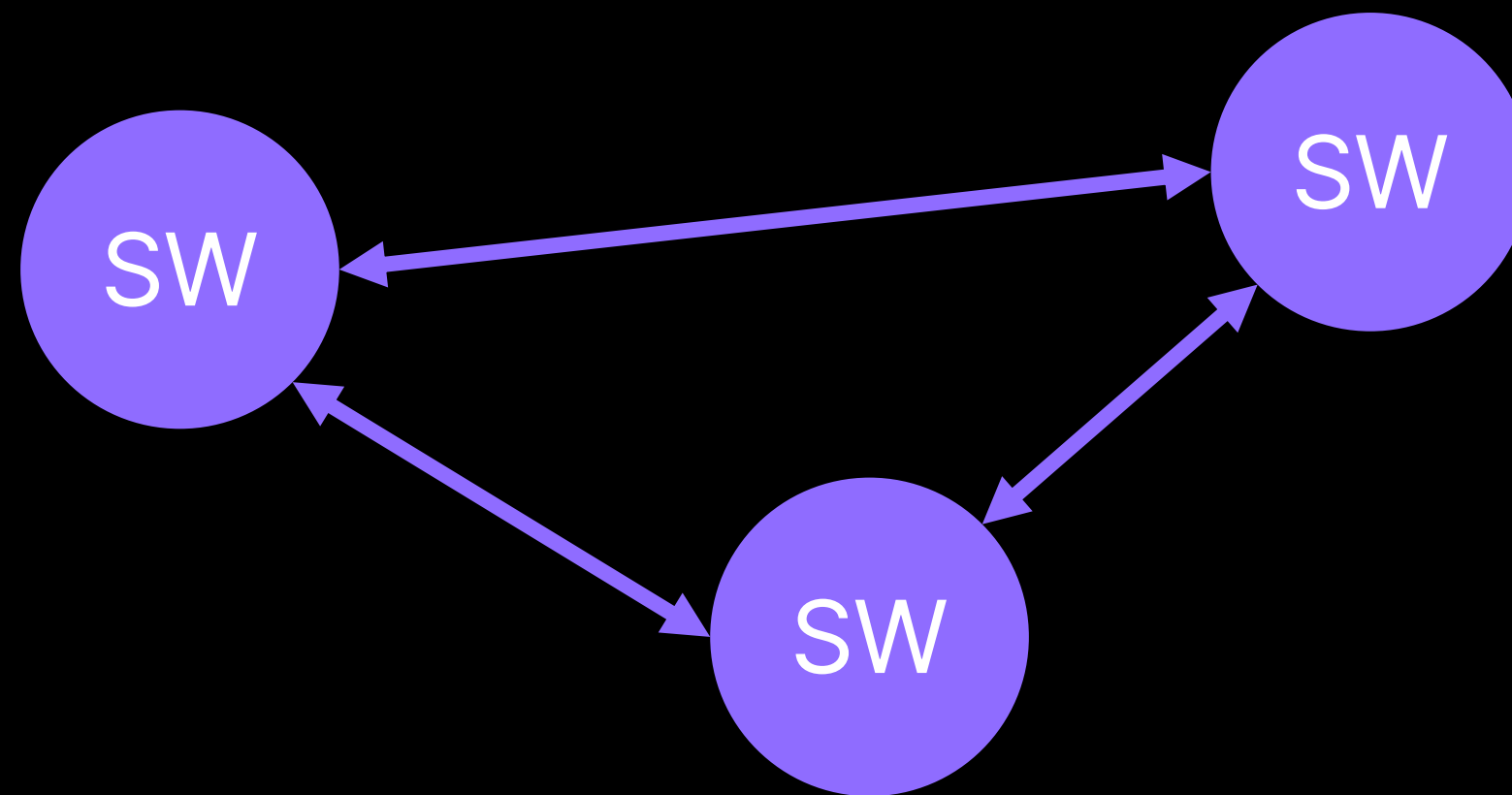


소프트웨어 애플리케이션 간의 상호작용을 가능하게 하는 인터페이스



API

Application **P**rogramming **I**nterface

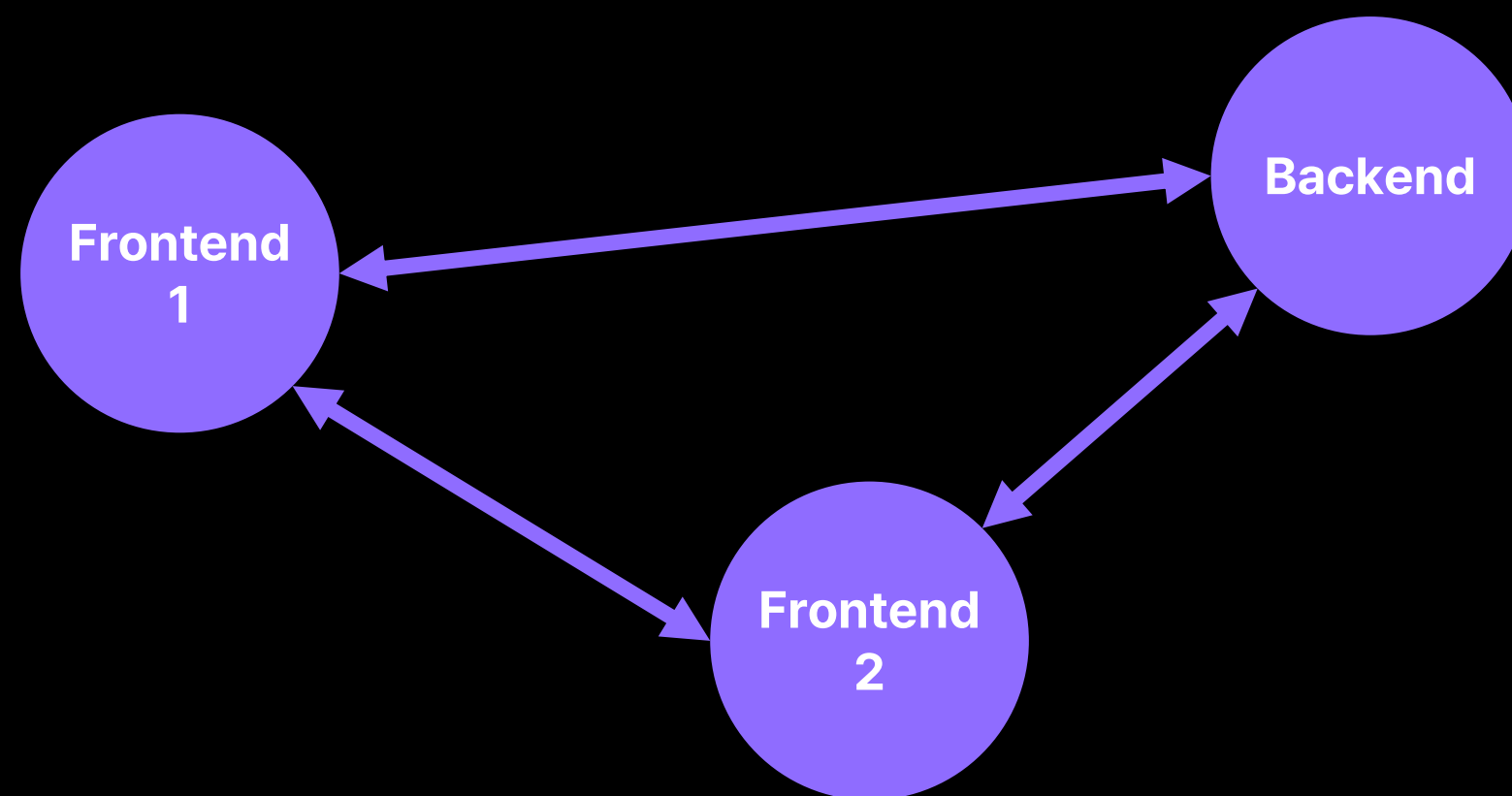


함수/메서드, 데이터 구조, 프로토콜, 라이브러리 등이 API가 될 수 있다.



API

Application Programming Interface



웹이라는 분야 안에서는 API는 주로

(HTTP를 기반으로) 웹 애플리케이션 간의 상호작용을 가능하게 하는 인터페이스를 의미한다.



REST

Representational **S**tate **T**ransfer

HTTP의 장점을 최대한 활용할 수 있는 아키텍처



REST API의 특징

1. 클라이언트-서버(Client-Server) 구조

- 사용자 인터페이스와 데이터 스토리지의 분리

2. 무상태(Stateless)

- 각 요청은 독립적, 서버에 상태 정보 유지 X

3. 캐시 처리 가능(Cacheable)

- 응답은 캐싱 가능해야함

4. 계층화 시스템(Layered System)

- 클라이언트는 서버를 직접 호출하지 않음(보안, 로드밸런싱)

5. 인터페이스 일관성(Uniform Interface)

- 효율적인 아키텍처를 위한 일관된 인터페이스

6. 코드 온 디맨드(Code-On-Demand)

- 서버는 실행 가능 코드 형태로 기능을 전송할 수 있음



REST API의 구성 요소

자원 (Resource)

관리되는 데이터나 객체

행위 (Verb)

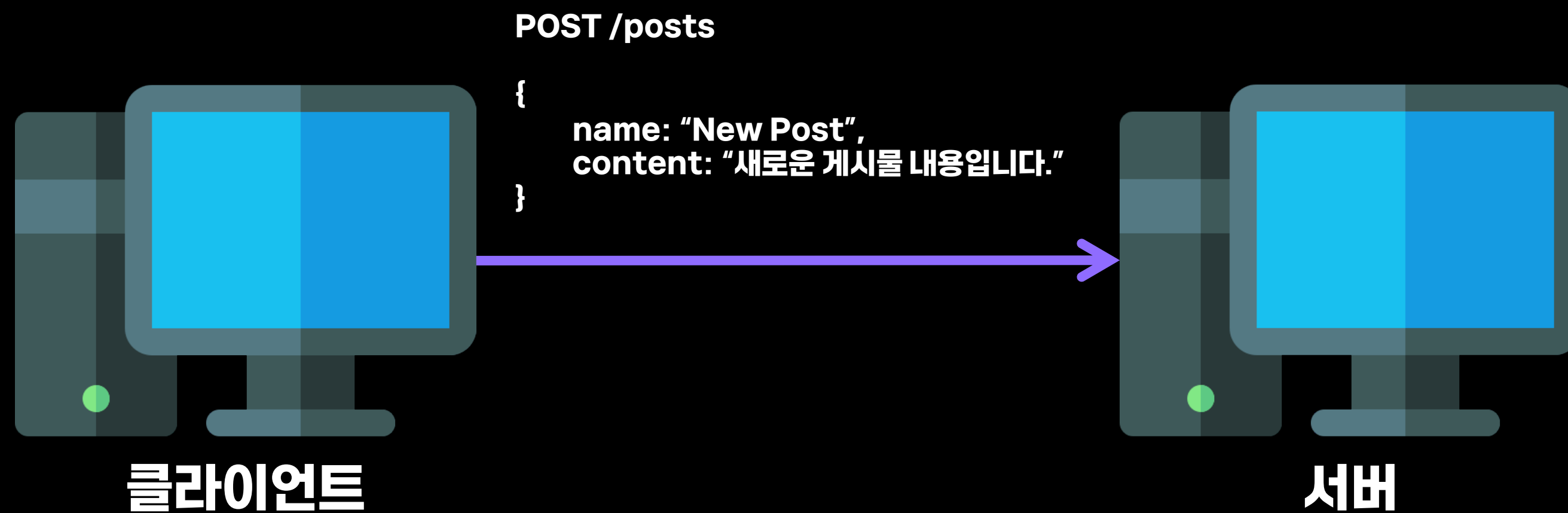
자원에 대해 수행되는 작업

표현 (Representations)

자원의 상태를 나타내는
데이터 형식



REST API의 구성 요소





REST API의 구성 요소





REST API의 구성 요소





REST API의 구성 요소

POST /posts

Body = 표현

```
{  
  name: "New Post",  
  content: "새로운 게시물 내용입니다."  
}
```




RESTful

REST 원칙을 잘 지킨 API 아키텍처를 RESTful하다고 이야기한다.



RESTful API가 되려면

GET /posts
POST /posts
GET /posts/1
GET /posts/1/comments

·
·
·

RESTful한 API를 설계하려면,

1. **자원을 중심으로 설계**해야 한다. 따라서 URI는 명사를 위주로 사용해야 한다.



RESTful API가 되려면

HTTP 메서드	역할
GET	데이터 조회
POST	데이터 생성
PUT	데이터 전체 교체
PATCH	데이터 일부 수정
DELETE	데이터 삭제

RESTful한 API를 설계하려면,
2. 자원에 대한 **행위는 HTTP 메서드로만 표현**한다.



Asynchronous JavaScript and XML

웹 페이지를 새로고침하지 않고도 서버와 비동기적으로 데이터를 주고받을 수 있게 하는 기술
현대의 브라우저에는 Ajax 기능이 모두 포함되어 있다.



XMLHttpRequest

```
// XMLHttpRequest를 사용한 AJAX 요청 예제
function loadData() {
    var xhr = new XMLHttpRequest();
    xhr.open('GET', 'https://api.example.com/data', true);

    xhr.onreadystatechange = function() {
        if (xhr.readyState === 4 && xhr.status === 200) {
            var data = JSON.parse(xhr.responseText);
            console.log(data);
        }
    };

    xhr.send();
}

// 함수 호출
loadData();
```

Ajax 기능을 활용하는 가장 기본적인 API는 `XMLHttpRequest` 객체이다.
그러나 사용이 매우 불편하다.



Fetch API

```
// Fetch API를 사용한 AJAX 요청 예제
function loadData() {
    fetch('https://api.example.com/data')
        .then(response => { return response.json(); })
        .then(data => { console.log(data); })
        .catch(error => { console.error(error); });
}

// 함수 호출
loadData();
```

Fetch API를 사용하면 **Promise**를 기반으로
훨씬 간단하게 Ajax 기능을 사용할 수 있다.



Fetch API

```
fetch('https://api.example.com/data');
```

‘fetch’ 함수의 첫 번째 인수로는 요청을 보낼 URL이 전달된다.



Fetch API

```
fetch('https://api.example.com/data');
```

두 번째 인수로는 `option` 객체가 전달되며,
없는 경우 기본으로 **GET 요청**을 보낸다.



Fetch API

```
fetch('https://api.example.com/data', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    name: 'Elice',  
    email: 'elice@example.com'  
  })  
});
```

‘option’ 객체에는 해당 요청의 Header, Body 등을 설정할 수 있는 여러 프로퍼티가 들어간다.



Fetch API

```
fetch('https://api.example.com/data', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    name: 'Elice',  
    email: 'elice@example.com'  
  })  
});
```

Body를 JSON 형태로 전달할 때에는
무조건 `JSON.stringify`를 통해 직렬화하여 전달해야 한다.



Fetch API

```
// Fetch API를 사용한 AJAX 요청 예제
function loadData() {
  fetch('https://api.example.com/data')
    .then(response => { return response.json(); })
    .then(data => { console.log(data); })
    .catch(error => { console.log(error); });
}

// 함수 호출
loadData();
```

‘fetch’는 Promise를 반환한다.

따라서 응답에 대한 이후 처리는 ‘then’, ‘catch’, ‘finally’의 메서드 체인을 통해 수행한다.



Fetch API

```
// Fetch API를 사용한 AJAX 요청 예제
function loadData() {
  fetch('https://api.example.com/data')
    .then(response => { return response.json(); })
    .then(data => { console.log(data); })
    .catch(error => { console.log(error); });
}

// 함수 호출
loadData();
```

‘fetch’ 호출 이후 반환하는 ‘Response’ 객체는
Body의 내용을 가공할 수 있는 여러 메서드를 제공한다.



Fetch API

```
// Fetch API를 사용한 AJAX 요청 예제
function loadData() {
  fetch('https://api.example.com/data')
    .then(response => { return response.json(); })
    .then(data => { console.log(data); })
    .catch(error => { console.log(error); });
}

// 함수 호출
loadData();
```

`.json()` 메서드를 사용하면 JSON 형태의 Body 내용을 간편하게 역직렬화해 준다.

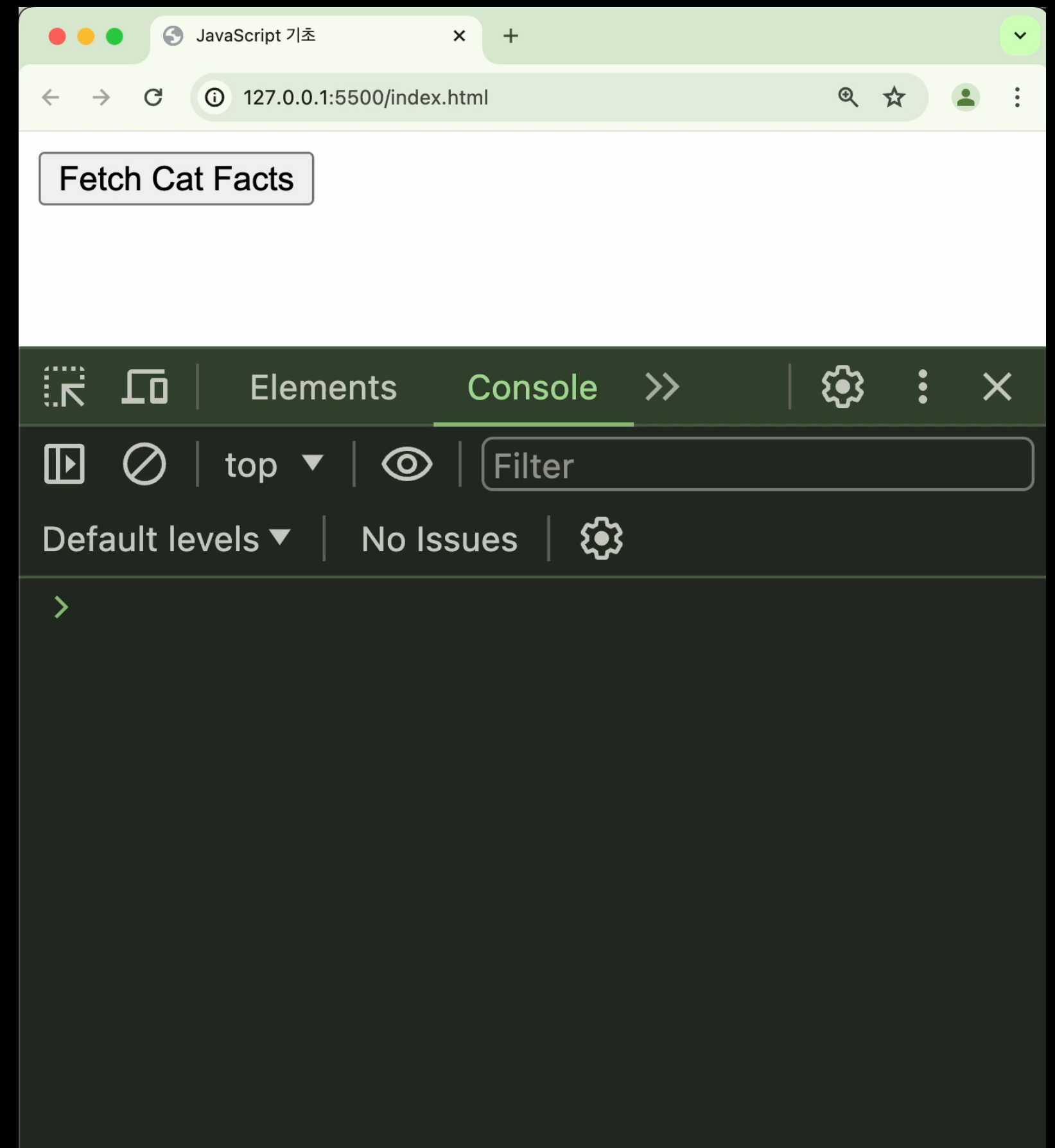
(해당 메서드 또한 Promise를 반환)



Fetch API 사용 예시

```
document.getElementById("fetchBtn").addEventListener("click", () => {  
  fetch("https://catfact.ninja/breeds")  
    .then((response) => {  
      console.log(response);  
      return response.json();  
    })  
    .then((data) => {  
      console.log(data);  
    })  
    .catch((error) => {  
      console.error(error);  
    });  
});
```

Play →



Fetch API 사용 예시 – async/await

```
document
  .getElementById("fetchBtn")
  .addEventListener("click", async () => {
    const response = await fetch("https://catfact.ninja/breeds");
    console.log(response);
    const body = await response.json();
    console.log(body);
  });
```

Play →

